

Frames, Signatures and Joints: an experimental study of exact, stochastic and GPU solvers for Eternity II-type edge-matching puzzles

Carlos Edison Guzman Marte — Calgary, Alberta, Canada (originally from the Dominican Republic)

October 2026 — DOI: 10.5281/zenodo.23132447 —
<https://github.com/Edison82do/eternity2-carlos-guzman>

Abstract

Eternity II is a 16×16 edge-matching puzzle with 256 pieces and 22 colours, released in 2007. It has never been solved. The best published partial result matches 470 of its 480 inner edges. This report documents a five-day line of independent research (29 September – 3 October 2026) on solvers for this class of puzzles. It covers three families of methods, each implemented, measured and timestamped. (1) The *frames and signatures* method: an exact backtracking solver that grew from a Python prototype into a multi-threaded C engine (versions 1 to 4.7). Its main ingredients are corner groups, frame signatures, side and row “maps” (arc consistency along whole lines), interior-first ordering chosen by Knuth’s tree-size estimator, portfolios of strategies with Luby restarts, and value ordering by belief propagation. (2) The *joints system* (version 5): a stochastic method that keeps an always-assembled synthetic board and gradually introduces the real pieces, with an exact “closure” for the last cells. (3) A *GPU line* (version 6): simulated annealing, exhaustive search and a port of the version-4.6 pruning, on a consumer graphics card. On medium boards (7×7 to 11×11) the exact solver went from minutes or no solution to fractions of a second. Examples: an 11×11 board dropped from 206.8 s to 0.34 s, and five 9×9 boards with five clues dropped to 33 s in total. The real puzzle remains far out of reach. We report the positive results and, just as carefully, the ideas that did not work, with the numbers that rule them out.

- [1. Introduction](#)
- [2. Problem, notation and benchmarks](#)
- [3. The frames-and-signatures method \(versions 1–3\)](#)
 - [3.1 The idea \(version 1, Python\)](#)
 - [3.2 Group by group, and side signatures \(version 2\)](#)
 - [3.3 C engine \(version 3\)](#)
- [4. Unified search \(versions 4.0–4.7\)](#)
 - [4.1 Side maps and a single search \(v4.0\)](#)
 - [4.2 Dynamic work sharing \(v4.1\)](#)
 - [4.3 Second-ring and row maps \(v4.2; the author’s idea\)](#)
 - [4.4 Knuth estimates and interior-first ordering \(v4.3\)](#)
 - [4.5 Portfolios and restarts \(v4.4\)](#)
 - [4.6 Joined corners \(v4.5\) and lazy maps \(v4.6\)](#)
 - [4.7 Belief propagation as value ordering \(v4.7\)](#)
- [5. Explorations and negative results](#)
 - [5.1 Extra pruning rules](#)
 - [5.2 SAT encoding](#)
 - [5.3 How many frames are there?](#)
 - [5.4 The designer’s fingerprint](#)
 - [5.5 Beliefs inside the search, and \$2 \times 2\$ block beliefs](#)
- [6. The joints system \(version 5\)](#)
 - [6.1 Idea \(the author’s\)](#)
 - [6.2 Exact closure](#)
 - [6.3 False near-solutions](#)
 - [6.4 Ideas tested and discarded](#)
 - [6.5 The real puzzle and the record](#)
- [7. The GPU line \(version 6\)](#)
 - [7.1 Simulated annealing on the GPU](#)
 - [7.2 Exhaustive search on the GPU](#)
 - [7.3 The v4.6 pruning on the GPU](#)

- [7.4 Belief propagation on the GPU](#)
- [8. Discussion](#)
- [9. Reproducibility and timestamps](#)
- [10. Authorship and AI-assistance statement](#)
- [References](#)

Authorship and use of AI. The method, the ideas and the decisions are the author’s. The code, the measurements and the write-ups were produced with the help of AI tools (Anthropic’s Claude), working under the author’s direction. All experiments were run either on the author’s own computer or in a cloud sandbox, and every solution reported as found was checked by an independent verifier. Section 10 gives details.

1. Introduction

The puzzle. Eternity II (E2) has 4 corner pieces, 56 border pieces and 196 interior pieces.

- Each piece has four coloured edges; the outside of the board is grey.
- Five colours appear only between border pieces, and seventeen only in the interior.
- One mandatory clue fixes a starting piece near the centre. Four optional clues come from smaller companion puzzles.
- The best known result keeps all 256 pieces on the board and matches 470 of the 480 inner edges. It was found by Joshua Blackwood in 2021 and equalled by Jef Bucas in 2024.

This work. The study was driven by a single question: *can an exact search, organised around the frame of the board, be pushed to the point where large instances become feasible?* Along the way it produced three distinct solver families, summarised in Table 1. All of them were built and measured on the same benchmark sets.

Line	Versions	Idea	Status
Frames and signatures	v1 – v4.7 (29 Sep – 2 Oct)	Exact backtracking organised by the frame	Strongest method; solves medium boards
Joints system	v5.0 – v5.4 (1 – 2 Oct)	Always-assembled synthetic board; introduce real pieces	Competitive up to 8×8; stalls on “false near-solutions”
GPU	v6 (3 Oct)	Annealing, exhaustive search and v4.6 pruning on a GTX 1070	Good for simple massive work; pruning stays on the CPU

Table 1. The three lines of work.

Contributions.

1. A complete description of the frames-and-signatures method and its evolution, with the measured effect of every change (Sections 3–4).
2. A set of measured negative results, each with the numbers that rule it out:
 - colour parity;
 - Hall-type matching;
 - SAT encodings;
 - recomputed beliefs and 2×2 block beliefs;
 - a heavy pruning search on the GPU (Section 5 and Sections 6–7).
3. The joints system and its main finding: the stochastic method gets trapped in “false near-solutions” from 9×9 upwards (Section 6).
4. Measurements of the frame count of 9×9 and 10×10 boards, and an analysis of the official puzzle’s design “fingerprint” (Section 5).
5. All code, logs and a chain of SHA-256 hashes timestamped on the Bitcoin blockchain through OpenTimestamps (Section 9).

2. Problem, notation and benchmarks

Notation.

- An instance is an $n \times n$ board and a multiset of n^2 square pieces, each given as four colours (N, E, S, W).
- Colour 0 is the grey border. A piece with two grey edges is a corner; one grey edge, a border piece; none, an interior piece.
- A solution places every piece exactly once, with a rotation, so that every pair of adjacent edges has the same colour and every grey edge faces outwards.
- *Clues* (“pistas”) are pieces fixed in position and rotation.
- Boards are described as $n \times n$ $I:B$: I interior colours and B border colours.

Benchmark families. Boards marked as verified-solvable come with a known solution.

Family	Description
gen_nxn_ck	Random boards with K colours shared between border and interior; 5×5 to 8×8 .
Editor models	The models shipped with Eternity Editor 1.6.0 (e.g. 0706 = 7×7 with 6 colours, 1116 = 11×11 with 16 colours).
Harris-type	Boards with few border colours and separate interior colours (6×6 6:2, 7×7 6:4, 8×8 7:2, 7:3, 7:4, 8:2), generated by our own scripts in the style of a benchmark family known in the community by that name (files harris_*). These are the hard ones for frame-based methods.
Known-solution boards	c9x9 9:3, c10x10 10:3, c12x12 14:4, generated with a stored solution. Clues follow the E2 pattern: $p1$ = centre only; $p5$ = centre plus four pieces near the corners.
“Official-type” boards	Like the real puzzle: exactly balanced colours, no duplicate pieces, no symmetric pieces (Section 5.4).

Hardware. Two machines were used, as stated in each experiment:

- the author’s PC: AMD Ryzen 5 3600 (6 cores, 12 threads), NVIDIA GeForce GTX 1070 (8 GB), Windows;
- a 2-core Linux cloud sandbox.

Times are wall-clock. Where stated, the ≈ 3 s spent on the Knuth estimate (Section 4.4) is excluded.

3. The frames-and-signatures method (versions 1-3)

3.1 The idea (version 1, Python)

1. **One corner fixed.** The first corner always goes top-left. This removes the four rotated copies of every solution without losing any.
2. **Corner groups.** The other three corners are permuted. That gives 6 groups, or fewer when some corners are identical.
3. **Frames.** For each group, every complete frame (the whole border ring) is generated, enumerating by piece *type* so that identical pieces never yield duplicate frames.
4. **Signatures.** The interior only “sees” the sequence of colours that the frame shows inwards: its *signature*. Many frames share a signature, and they count as a single case for the interior.
5. **Interior search.**
 - Always fill the cell with the fewest options (MRV).
 - After each placement, check that no cell and no piece is left without a place.
 - Cells next to the frame accept only colours that occur in some still-alive signature. All frames of all groups are applied at once, as one bitwise AND.

6. **Independent verifier.** Every solution is checked for borders, colours and the exact multiset of pieces.

Comparison with an existing solver. The prototype was compared with the *Iterative Path MkIV* solver of Eternity Editor (Java) on the same boards, with a 60 s limit (Table 2).

Board	Frames v1 (Python)	MkIV (Java)
5×5, 6 colours (10)	10/10, median 0.00 s	10/10, 0.05 s
6×6, 6 colours (10)	10/10, 0.01 s	10/10, 0.15 s
7×7, 6 colours (10)	7/10, 5.2 s	5/10, 52.1 s
8×8, 8 colours (5)	0/5	0/5
Model 0706 (5 shuffles)	0/5	4/5, 20.8 s

Table 2. Version 1 against MkIV (60 s limit; unsolved runs count as 60 s).

Where v1 failed. The method failed when there were too many frames: between about 38 000 (model 0706) and 2.7 million (one 7×7 board).

3.2 Group by group, and side signatures (version 2)

Group by group (the author’s idea). Groups are ordered from cheapest to most expensive, and solved one at a time (frames, signatures, interior) with all cores. If a group has no solution, its signatures are *removed* from the following groups. This is sound because the interior depends only on the signature. It was checked on 150 boards with no error.

Side signatures. Each side of the frame is treated as a chain of border pieces from one corner to the next. There are far fewer chains than frames. The only thing that links the four sides is that no piece may be used twice, and this is checked when preparing and again during the search.

Effect. Group by group was the largest single gain of the early versions:

Board	v1	v2, 1 core	v2, 2 cores
Model 0706	0/5	5/5, 17 s	5/5, 8.8 s
8×8	0/5	3/5	4/5

3.3 C engine (version 3)

The same method was rewritten in C, with three additions:

- 64-bit signature filters;
- skipping of regions where no signature is alive;
- multiple threads.

The speed-up over v2 was 18–60×. Table 3 shows the effect.

Board	MkIV	v2, 1 core	v3, 1 thread	v3, 2 threads
7×7 (10)	5/10 (52 s)	7/10 (10 s)	10/10 (0.36 s)	10/10 (0.24 s)
8×8 (5)	0/5	3/5 (58 s)	4/5 (0.26 s)	4/5 (0.14 s)
Model 0706 (5)	4/5 (21 s)	5/5 (17 s)	5/5 (0.93 s)	5/5 (0.55 s)

Table 3. Version 3 (cloud, 2 cores, 60 s; medians in brackets).

Limit of v3. With few border colours, as in Harris-type boards or the real E2, a single group has tens of millions of frames, or too many to count. On the real E2, one side alone exceeded 10 million chains.

4. Unified search (versions 4.0–4.7)

4.1 Side maps and a single search (v4.0)

Side maps. Instead of listing chains, each side becomes a *map*: which colour can sit on each joint and which piece fits in each cell. It is pruned with a forward pass and a backward pass, so the full list is never stored.

Counting by piece type. Identical pieces are grouped into types and counted from the start. When a type runs out, it disappears from every cell. If a type has fewer possible cells than copies left, the branch is cut.

A single search. Border and interior are decided in the same search, always taking the most constrained cell (MRV). A decision on the border immediately restricts the interior, and vice versa.

Clues. Clues are supported (-P row,col,piece,rotation). On the real E2 with the mandatory centre clue, v4.0 placed up to 212 of 256 pieces with every edge matching in 60 s (2 cores). Version 3 could not even start on that board.

4.2 Dynamic work sharing (v4.1)

In v4.0, the branches of the tree were split among the threads only once, at the start of each group. Threads with short branches went idle; on an 11×11 board, speed fell from 3.2 to 0.5 M nodes/s.

Version 4.1 uses a shared task queue instead. A busy thread checks every 256 nodes whether another thread is idle (“hungry”), and if so gives away its untried options at its shallowest level.

Completeness check. 48 boards were solved with 1 and with 16 threads, and the results and the group where the solution appeared were compared: they matched in all 48 cases. This check caught a real bug during development: giving away work corrupted the path of the giving thread.

4.3 Second-ring and row maps (v4.2; the author’s idea)

The idea. The inner ring of the board also filters the outer frame. Enumerating inner frames would be hopeless, so each side of the second ring is treated as a map, exactly like the border sides:

- forward/backward passes delete the orientations that belong to no compatible complete row;
- the colours the ring can still show outwards restrict the neighbouring border cells;
- those border cells feed back into the side maps.

With `--anillo2 2` the same is applied to **every** interior row and column. Table 4 shows the effect.

Mode (11×11, 2 threads)	Group 1 (no solution)	Group 2 (no solution)	Total to solution
no maps (as v4.1)	94 s, 94 M nodes	53 s, 56 M	203 s, 205 M
second ring only	42 s, 11 M	44 s, 11 M	121 s, 31 M
all rows and columns	26 s, 1.8 M	38 s, 2.4 M	98 s, 6.8 M

Table 4. Row maps on the 11×11 Editor model (groups without a solution give the cleanest comparison).

Trade-off.

- The maps cut nodes by about 30×, but each node costs about 3× more.
- On small boards ($\leq 8 \times 8$) they cost more than they save.
- On the author’s PC (12 threads), Harris 8×8 7:2 s01 went from 223 s to 49 s, and the 11×11 from 61.6 s to 20.3 s. From v4.3 on, the maps are always on.

4.4 Knuth estimates and interior-first ordering (v4.3)

Knuth's (1975) random-probe estimator is run per group before searching. It turned out to be very accurate on groups without a solution: on the 11×11 it predicted 2.0 M and 2.7 M nodes, and the actual counts were 1.9 M and 2.5 M.

The estimator is used to choose, per group, between two cell orders: “fewest options” and “**interior first, border last**”. Interior first is chosen only if its tree is at least 3× smaller.

Interior first made a dramatic difference on boards with few border colours, where the border hardly restricts anything even though its cells have few options:

Board (PC, 12 threads)	Fewest options (v4.2)	Interior first	Speed-up
Harris 8×8 7:2 s01	49 s, 65 M nodes	3.2 s, 4.3 M nodes	15×
Harris 8×8 7:2 s02	152 s, 204 M nodes	4.7 s, 5.6 M nodes	32×

On the 9×9 9:3 board, the estimated tree went from 4.3×10^{15} to about 9×10^{13} nodes, roughly 50× smaller.

4.5 Portfolios and restarts (v4.4)

The problem. On the 16 hardest Harris 8×8 boards, each single strategy solved about 9 of 16 within 5 minutes, but not the same 9. This is a classic heavy-tailed behaviour.

Automatic portfolio. When the Knuth estimate of a group exceeds 10^9 nodes, three teams of threads search the same group:

1. the normal complete search;
2. a complete search that tries the least-constraining piece first;
3. restarts with randomised value order and a node cap that grows by the Luby sequence.

The first team to find a solution, or to exhaust the group, wins. Coverage of the hard set went from 9 to 11 of 16.

Fix in the same version (the author's observation). With a clue on the board, the board can no longer be rotated, so the top-left corner cannot be fixed. Versions 4.0–4.3 could wrongly answer “no solution” in that case: they failed in 4 of 4 test cases. Version 4.4 tries all four corners top-left and solves 144 of 144 cases: 48 boards × 3 rotations, with the centre piece fixed.

4.6 Joined corners (v4.5) and lazy maps (v4.6)

Joined corners (v4.5). Up to v4.4, the interior search was essentially repeated for every corner group. In v4.5 the four corners become ordinary cells of the search, with their options filtered by the side maps. The interior is searched once for all groups. Without clues, one corner remains fixed, so that rotated copies of the board are not searched.

Lazy maps (v4.6). With all pieces distinct, placing a piece removes it from every free cell. Version 4.5 took that as a change in every row and recomputed every map. Version 4.6 recomputes a row or column only if one of its cells changed for another reason. This doubled the nodes per second with an identical tree: the Knuth estimates did not change.

Board (PC, 12 threads)	v4.0	v4.1	v4.2	v4.3	v4.4	v4.5	v4.6
11×11 Editor model 1116	206.8 s	61.6 s	20.3 s	23.5 s	27.9 s	0.6 s	0.34 s
Harris 8×8 7:2 s01	—	230 s	49 s	3.2 s	5.7 s	1.4 s	1.2 s
Hard Harris 8×8 solved in 5 min (of 16)	—	—	—	9	11	14	13
9×9 9:3 with 5 clues (5 boards)	—	—	—	(wrong “no solution”)	8 s - 6 min	1 s - 88 s	0.03 s - 95 s

Table 5. Evolution of the exact solver on the author’s PC. The 14 → 13 difference on the hard Harris set is run-to-run noise from the portfolio (heavy tails): summed over the 13 boards solved by both versions, v4.6 needed 576 s and v4.5 932 s.

Completeness. Every version from v4.1 on was checked against v4.2 on 48 boards, with 1 and with 8 threads. All 96 runs agreed (cmp_grupos.txt in each folder).

4.7 Belief propagation as value ordering (v4.7)

How the beliefs are computed. creencias.py runs loopy belief propagation over edge colours:

- each cell exchanges messages with its neighbours about the colours of the shared edges;
- Sinkhorn balancing makes each piece used about once;
- the result is a probability for every (piece, rotation) in every cell.

The C engine then tries the most probable options first: --valor 3 for all threads, or --portafolio 7, which runs one complete team in belief order and one team of belief-biased restarts.

Results. On the five 9×9 9:3 boards with five clues (12 threads):

Board	Automatic (= v4.6)	Beliefs only	Beliefs + restarts
s1	101.6 s	3.3 s	22.7 s
s2	3.7 s	3.5 s	3.3 s
s3	3.1 s	3.1 s	3.1 s
s4	33.4 s	17.1 s	35.0 s
s5	8.0 s	5.9 s	3.3 s
Total	150 s	33 s	67 s

Table 6. Beliefs as value ordering (about 3 s of each time is the Knuth estimate).

Limits.

- Without clues the beliefs are useless: the board is symmetric and every cell looks the same.
- On 10×10 10:3 boards with five clues, no configuration found a solution within 30–60 minutes. The furthest any search got was 76 of 100 pieces placed.

5. Explorations and negative results

5.1 Extra pruning rules

The following rules were each measured with the Knuth estimate on the 9×9 9:3 board or by solving the 16 hard Harris boards:

- colour parity;
- piece counting;
- forcing pieces that fit in only one cell (the “inverse” view, an idea of the author);
- maps iterated to a fixed point;
- a global (Hall-type) matching check.

They cut between 0 % and 30 % of the nodes, but each node becomes more expensive. They remain available as options.

Two other changes did not help either:

- branching on pieces instead of cells was worse;
- a row-by-row interior order shrank the 9×9 estimate 2.3× but did not find the Harris solutions earlier.

5.2 SAT encoding

A change of representation to SAT, solved with CaDiCaL and Glucose, was 10 to 100 times slower than the C engine on the same boards.

5.3 How many frames are there?

Knuth estimates over 40 000 random paths, with the top-left corner fixed. Here “ring possible” means the first interior ring can still be built with interior pieces.

	Valid frames	Distinct signatures	Frames per signature (median)	Signatures with a possible ring
9×9 9:3	$\approx 9.5 \times 10^{16}$	$\approx 1.5 \times 10^{15}$	48	$\approx 2.2 \times 10^{14}$ (14 %)
10×10 10:3	$\approx 6.0 \times 10^{20}$	$\approx 5.7 \times 10^{19}$	8	$\approx 3.2 \times 10^{19}$ (59 %)

Table 7. Frames and signatures.

Reading.

- Signatures reduce the frames 63× on 9×9, but only 10× on 10×10: the reduction shrinks as boards grow.
- Going frame by frame, even grouped by signature, is more expensive than the unified search.
- This explains why the method moved from enumerating frames (v1–v3) to the unified search (v4).

5.4 The designer’s fingerprint

The official piece list is sorted by type (corners, border, interior) and, within each type, by colour. The numbering carries no information about the solution.

Compared with 300 random boards built with the same parameters, the official set follows three rules exactly:

- no duplicate pieces;
- no symmetric pieces;
- perfectly balanced colours: every border colour on exactly 12 edges, every interior colour on 24 or 25.

Official-type boards. A generator for boards that follow the same rules was written. Such boards are *not* harder on average than random ones, but much less variable. For example, on 9×9 9:3 the Knuth estimates were:

Boards	Geometric mean	Range
Random	$10^{15.4}$	$10^{14.3} - 10^{16.8}$
Official-type	$10^{15.0}$	$10^{14.7} - 10^{15.4}$

No further fingerprint. Against 200 official-type 16×16 boards, the official puzzle falls in the normal range on six further statistics. Examples:

- colour pairs inside a piece: 46th percentile;
- repeated border pairs: 29th percentile.

No weakness of the kind exploited in Eternity I was found.

5.5 Beliefs inside the search, and 2×2 block beliefs

How much information the beliefs carry. On a 10×10 board with five clues, the beliefs become exact once about 20 random correct pieces, or 30 frame pieces, are fixed.

They do not separate good prefixes from bad ones.

- They cannot tell a correct 20-piece row-major prefix from a wrong one.
- They can with a 30-piece frame prefix, but at that point the exact search also refutes wrong prefixes immediately: with 0 nodes, in the case of v4.6.
- Recomputing beliefs inside the search therefore adds no pruning that the exact search lacks.

Beliefs over 2×2 blocks. Each block admits only combinations of four distinct pieces that match each other. The variant improved the median rank of the true piece only slightly (10×10 s2: from 10 to 7).

On the author’s PC, version 4.7 was run on the 10×10 10:3 boards with five clues for 30 minutes per configuration. With 2×2 beliefs instead of 1×1 beliefs, it reached the same maximum depth in every case: 76/100 on s1 and 73/100 on s2.

6. The joints system (version 5)

6.1 Idea (the author’s)

1. Start from a synthetic, **already assembled** board with the same number of edges of each colour as the target.
2. Introduce the real pieces one by one, without ever disassembling the board.
3. Corners keep their real colours but may move among the four corners.
4. Clues are fixed. Without clues, one corner is fixed.

Representation. The program does not place pieces: it chooses the **colour of each joint** (inner edge). The board is therefore always assembled, and the colour proportions never change, because colours are only swapped between joints. A cell is *real* if the piece formed by its four joints exists in the set (counting copies). The goal is all cells real, which **is** an exact solution.

Moves and search.

- Two moves: swapping two joint colours, and “introducing” a missing real piece into a fictitious cell.
- Search by simulated annealing, then parallel tempering on 12 threads (temperature ladder 0.15–0.8).
- The C engine runs about 1.5 M steps/s per thread.

6.2 Exact closure

When annealing leaves few fictitious cells, the region around them (radius 1, then 2) is rebuilt **exactly**. The real pieces outside the region stay where they are, and the region is

filled by an MRV backtracking search with a node budget.

On the author's PC with 12 threads, the closure turned results like these into solutions:

Board	Annealing only	With closure
gen 7×7 s2	stuck at 48/49	3.7 s
gen 8×8 s1	stuck at 60/64	2.1 s
gen 8×8 s2	stuck at 60/64	4.4 s

On generated 8×8 boards this is as fast as or faster than v4.6. On Harris 8×8 7:2 boards it solved 4 of 10, beating v4.6 on one (s04: 7.1 s against 8.7 s); v4.6 solved all 10.

6.3 False near-solutions

The key finding of this line. On a 9×9 9:3 board with a known solution, the best annealed board had **74 of 81 real pieces, but only 7 in their correct place**. The stochastic search finds almost complete boards that are far from the true solution, so a local closure cannot finish them.

Control experiment. Given the correct frame as clues, the interior of the 9×9 is solved in 0.08 s, by this system and by v4.6 alike. For both systems, the hard part is the frame.

6.4 Ideas tested and discarded

None of the following broke the false near-solutions:

Idea	Result
Exact window repair (3×3 or 5×5 blocks keeping colours)	Fit in 2 of 2 600 attempts
Lagrange-style piece prices	74/81 real, but only 4 in place
Guided local search	73/81 real, only 5 in place
“Skeleton” analysis of 40 near-solutions	Agreement only next to the fixed corner
Static (fixed) order in the exact closure	Much worse than dynamic MRV

Only belief propagation brought new information. That observation led to version 4.7.

6.5 The real puzzle and the record

Converting to the standard score. The joints system can report the standard score: keep the real pieces, put the missing pieces into the fictitious cells with their best rotation, and count matching edges. Conversely, a board with 470 matching edges can be represented with at most 10 fictitious cells (one per mismatched edge), so 470/480 corresponds to at least 246 real pieces of 256.

Runs on the official puzzle (five clues).

Run	Real pieces	Standard score
PC, 15 s	220/256	411/480
Author's long run, 7.7 h	233/256	435/480
Starting from the record board (centre clue only)	rose to 248	fell to 461

The last row shows that the two objectives — real pieces and matched edges — are not aligned.

7. The GPU line (version 6)

All runs in this section used the author's GTX 1070, programmed with CuPy and NVRTC; CPU parts ran on the 12-thread Ryzen.

7.1 Simulated annealing on the GPU

Method. Real pieces are always on the board; pieces are swapped and rotated to maximise matching edges. Up to 30 720 boards run at once on a temperature ladder (0.15-1.0, 32 steps), each board kept in shared memory. The speed is about 750-840 M steps/s. The CPU performs the exact closure.

Results.

- Generated 7×7 and 8×8: 6/6 (2-37 s).
- Harris 8×8: 0/3, stopping at 110/112 edges.
- 9×9 with five clues: 137/144 edges.

The method is fast but gives no guarantee.

7.2 Exhaustive search on the GPU

Method.

- A row-major backtracker with candidate tables indexed by (colour above, colour to the left) and a bitmask of used pieces.
- The CPU cuts the tree into 200 000 to 2 000 000 *prefixes* (partially filled boards).
- Each GPU thread exhausts its prefixes, keeping resumable state between short kernel launches.
- If every prefix is exhausted without a solution, that **proves** there is none.

Kernel versions. Three were measured on the 9×9 board:

Version	What it adds	Speed (G nodes/s)
1	per-thread local memory	0.52-0.56
2	board data in shared memory	0.78-0.99
3	plus the used-piece masks in shared memory	1.32-1.39

On 10×10 boards the speed reached 1.87 G nodes/s. The same algorithm on 12 CPU threads runs at 0.29 G nodes/s.

Board	GPU exhaustive search	Reference
Harris 8×8 7:2 s01-s10	10/10 , 0.4-29 s	v4.6: 10/10, 3-16 s each
9×9 9:3 with five clues, s1-s5	62 / 8 / 9 / 157 / 6 s ($\approx$ 240 s in total)	v4.6: 150 s; v4.7 with beliefs: 33 s
10×10 10:3 with five clues	not solved in 30 min (best 85/100)	estimated full tree $\approx 10^{15}$ nodes

Table 8. GPU exhaustive search.

Why v4.6 still wins on 9×9. Raw speed is not enough. On 9×9 s1 the plain row-major search needed 8.6×10^{10} nodes. Judging from its run time and node rate, v4.6 needs on the order of 10^8 .

7.3 The v4.6 pruning on the GPU

What was ported. The essentials of v4.6:

- per-cell domains as bitsets;
- MRV with interior first;
- removing a placed piece from every other cell;
- neighbour filtering;
- lazy maps for every row and column;
- a check that every remaining piece still fits somewhere.

It is correct and solves boards. On 9×9 s2 with five clues, the same code needed 8.3 s on the CPU (12 threads), against 3.7 s for v4.6 itself.

Speed on the GPU. It processed only about 0.1 M nodes/s, ten times fewer than the same code on 12 CPU threads (1 M nodes/s). Neither 2 048-16 384 threads nor an interleaved, coalesced memory layout changed this.

Reason and lesson. Each node is a long, branch-heavy sequence of different work, which is exactly what GPU warps handle badly. The GPU is excellent for simple, massive work; heavy pruning belongs on the CPU.

7.4 Belief propagation on the GPU

A vectorised version runs on either device. The plain (1×1) beliefs take about 3 s for 16×16 even on the CPU, so the GPU brings no benefit there. The 2×2 beliefs would need the GPU's memory at 16×16 , but they did not improve the search (Section 5.5).

8. Discussion

What carried the exact method. On every board family, most of the speed-up came from three changes:

1. pruning that sees whole lines: side maps (v4.0) and row maps (v4.2);
2. measuring before searching: Knuth estimates choosing interior first (v4.3);
3. not repeating work: group by group with signature elimination (v2), and joined corners (v4.5).

Portfolios and beliefs help mainly on boards that do have a solution, by finding it earlier, because value ordering cannot shrink a tree that has to be exhausted.

Why the real puzzle is out of reach. The measurements consistently point at the frame:

- frames and signatures grow from about 10^{15} (9×9) to about 10^{19} (10×10);
- with the correct frame given, the interior of a 9×9 is solved almost instantly by both the exact and the stochastic methods;
- the stochastic methods find many near-complete boards that are far from any solution.

Already at 10×10 with five clues, the best exact configurations were estimated at hours to days, and at 12×12 at years.

Honest limits. Several conclusions rest on few boards, and run-to-run variance is large on heavy-tailed instances. Every reported solution was verified, and every completeness claim was checked against earlier versions. Timing comparisons with external solvers were made on the same machine but against different implementations and languages.

9. Reproducibility and timestamps

What is published.

- The source of every version, with its original documentation in Spanish.
- The benchmark boards and generators.
- The test logs from the author's PC, and Windows binaries of the final versions.

The official Eternity II piece list and the clue puzzles are **not** redistributed. The solvers read any board in the documented text format.

Timestamps. Each version was packed into a zip with a manifest of per-file SHA-256 hashes. The SHA-256 of each zip was registered on the dates below and timestamped with OpenTimestamps. The .ots proofs are in `registry/timestamps/`. The zip files themselves are kept by the author.

Package	Registered (Calgary time)	SHA-256
v1-v2	2026-09-29 19:44	0519e2b2...c1f42a
v3	2026-09-30	e5eeb785...65261b
v4.0	2026-09-30 09:55	c05bbda9...bc49540
v4.1	2026-09-30 10:39	c3ac611c...f7f3
v4.2	2026-09-30 13:49	70f9ca3c...6f21
v4.3	2026-09-30 23:41	17121efd...90c2
v4.4	2026-10-01 15:29	5203cea5...7bd5
v4.5	2026-10-01 19:53	3d2324d2...5a84
v4.6	2026-10-01 23:15	4f6eb419...e940
v4.7	2026-10-02 22:18	2369ee79...c877d
Joints system v5	2026-10-02 22:18	0f2b9911...f252
GPU v6	2026-10-03 12:40	87b5be96...c20f

Table 9. Registered packages (full hashes in `registry/SHA256SUMS_registration_zips.txt`).

10. Authorship and AI-assistance statement

The author, Carlos Edison Guzman Marte, conceived the research programme and the methods. The ideas attributed to him in this report include:

- the frames-and-signatures method itself;
- solving group by group with signature elimination;
- the second-ring maps;
- the joints system;
- the correction for clues and fixed corners;
- the “inverse / contrary / complement” questions that led to several of the pruning experiments;
- the decision to measure every idea and to pursue only exact solutions rather than partial scores.

The implementation, the experiments and the drafting of the documentation were carried out with the assistance of AI tools (Anthropic’s Claude), under his direction and with his review. Decisions about what to try, what to keep and what to publish were his.

References

- D. E. Knuth. Estimating the efficiency of backtrack programs. *Mathematics of Computation* 29 (1975) 121–136.
- M. Luby, A. Sinclair, D. Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters* 47 (1993) 173–180.
- R. Sinkhorn. A relationship between arbitrary positive matrices and doubly stochastic matrices. *Annals of Mathematical Statistics* 35 (1964) 876–879.
- J. Pearl. *Probabilistic Reasoning in Intelligent Systems*. Morgan Kaufmann, 1988 (belief propagation).
- E. D. Demaine, M. L. Demaine. Jigsaw puzzles, edge matching, and polyomino packing: connections and complexity. *Graphs and Combinatorics* 23 (2007) 195–208.
- Eternity Editor 1.6.0 (Iterative Path MkIV solver), used for the early comparison.
- OpenTimestamps, <https://opentimestamps.org>.